

Bitwise Operators in Java

Directly manipulating the binary representation of data for maximum efficiency and system-level control.

The Mechanics of Bitwise Logic

Understanding the 0s and 1s that power modern computing.

| Binary Foundations

What are they?

Bitwise operators perform actions directly on the **binary (bit) representation** of integer data types like byte, short, int, long, and char.

Why use them?

They are essential for **low-level programming**, including system optimization, networking, cryptography, and embedded systems where every bit counts.

Java Bitwise Operators Overview

Operator	Name	Description	Example (5 & 3)
&	Bitwise AND	Result is 1 only if both bits are 1.	5 & 3 = 1
	Bitwise OR	Result is 1 if either bit is 1.	5 3 = 7
^	Bitwise XOR	Result is 1 if bits are different.	5 ^ 3 = 6
~	Bitwise NOT	Inverts all bits (unary operator).	~5 = -6
<<	Left Shift	Shifts bits left (multiplies by 2 ⁿ).	5 << 2 = 20

| Bitwise Logic Truth Tables



AND (&)

$0 \& 0 = 0$
 $0 \& 1 = 0$
 $1 \& 0 = 0$
 $1 \& 1 = 1$



OR (|)

$0 | 0 = 0$
 $0 | 1 = 1$
 $1 | 0 = 1$
 $1 | 1 = 1$



XOR (^)

$0 \wedge 0 = 0$
 $0 \wedge 1 = 1$
 $1 \wedge 0 = 1$
 $1 \wedge 1 = 0$

| The Bitwise NOT & Two's Complement

~5

RESULTS IN
-6

Why the change?

Java uses **Two's Complement** to represent negative numbers. The NOT operator (~) flips every bit (0 → 1, 1 → 0). When you flip the bits of 5 (0101), you get a pattern that represents -6 in signed 32-bit arithmetic.

| Understanding Shift Operators

Signed Shifts (<<, >>)

Left Shift (<<): Moves bits left, filling with 0s.
Effectively multiplies by 2^n .

Right Shift (>>): Moves bits right, preserving the sign bit.

Unsigned Shift (>>>)

Moves bits right and fills the leftmost bits with 0, regardless of the original sign. Essential for logical bit manipulation where sign isn't numeric.

High-Performance Applications

Moving beyond basic logic into real-world engineering optimizations.

Computational Optimization Tricks



Even or Odd

`(num & 1) == 0` is faster than using the modulo (%) operator as it only checks the LSB.



Fast Swap

Swap a and b using XOR without a temp variable: `a^=b; b^=a; a^=b;`



Power of 2

Check if a number is a power of 2 using:

```
(n & (n - 1)) == 0
```

| Real World: Permission Flags

- 🛡️ **Efficiency:** Storing permissions (Read, Write, Execute) as bits in a single byte instead of multiple booleans.
- 🔍 **Masking:** Use AND to check if a specific permission bit is set.
- ⊕ **Adding:** Use OR to grant a new permission flag.
- 🔄 **Toggling:** Use XOR to flip a specific user permission status.



| Top Interview Concepts

- ❓ **Logical vs Bitwise:** Explain why `&&` short-circuits while `&` always evaluates both operands.
- ❓ **Signed vs Unsigned:** When would you choose `>>>` over `>>`? (Hint: handling negative bitmasks).
- ❓ **Performance Gain:** Why shift operators are preferred in graphics and cryptography over standard math.
- ❓ **Two's Complement:** The reason behind $\sim x = -(x+1)$.

Questions?

Bitwise operators offer a window into the core of computing. Mastery of these tools is a hallmark of an advanced Java developer.

THANK YOU